

Agent Mesh

Build. Fund. Wire. Run.

The Visual Canvas for Autonomous AI Agent Networks on Algorand

Version	1.0
Date	April 2026
Track	Agentic Commerce (x402) • Best Devtool
Author	Soubhagya Levi • UEM Kolkata
Network	Algorand Testnet • GoPlausible x402

Abstract

AgentMesh is a visual drag-and-drop canvas for building, funding, and running autonomous AI agent pipelines on Algorand. It fills the critical gap between visual workflow design and on-chain agentic commerce — providing a no-code execution layer that combines a React canvas, a FastAPI orchestration runtime, and Algorand-native payment infrastructure in a single unified product.

This paper describes the architecture, design decisions, execution model, and Algorand-native primitives that make AgentMesh the first visual sandbox for autonomous agent commerce on any blockchain. It covers the node and wire model, single-agent and multi-agent execution paths, x402 payment integration, A2A inter-agent messaging, Algorand wallet lifecycle, and the pipeline deployment and API surface.

Contents

1	Introduction & Motivation
2	System Architecture
3	The Canvas: Nodes and Wires
4	Pipeline Lifecycle
5	Single-Agent Execution
6	Multi-Agent Execution
7	Algorand Integration
8	x402 Payment Protocol
9	Agent-to-Agent (A2A) Communication
10	Tool System
11	Security and Trust Model
12	Future Work

SECTION 1

Introduction & Motivation

The emergence of autonomous AI agents capable of transacting, reasoning, and coordinating represents one of the most significant shifts in software architecture in recent years. Yet building multi-agent systems today requires substantial engineering effort: custom orchestration code, manual wallet management, bespoke payment integration, and no standardized way to visualize or debug agent pipelines.

Algorand's native primitives — instant finality, sub-cent fees, atomic transfers, and the newly live x402 micropayment protocol — make it uniquely suited as the settlement and trust layer for agentic commerce. Yet as of April 2026, no visual tooling exists for building agent pipelines on Algorand. Developers must wire everything by hand.

AgentMesh closes this gap. It is the first canvas-based execution environment for autonomous agent networks on Algorand — combining visual design, real on-chain wallets, x402 payment flows, and A2A inter-agent messaging into a single deployable product.

The Problem Space

No visual tooling. Every agentic Algorand project is built from scratch. No canvas, no drag-and-drop, no fast iteration.

Agents are invisible on-chain. No standard way to spin up an agent with a real Algorand wallet, fund it, and observe it transact.

No wiring layer. Connecting agents to other agents or x402 services requires custom HTTP plumbing every single time.

Visual builders lack blockchain. Tools like n8n, Flowise, and Dify exist for web2 workflow automation but have no on-chain execution or payment primitives.

SECTION 2

System Architecture

AgentMesh is composed of three tightly integrated layers:

Layer	Technology	Responsibility
Canvas (Frontend)	React + ReactFlow + TypeScript	Visual workflow design, node/wire management, deploy & r
Runtime (Backend)	FastAPI + Python + asyncio	Pipeline orchestration, agent microservices, tool execution,
Payment Layer	Algorand + py-algorand-sdk + GoPlausible + Wallet	Wallet creation, balance polling, x402 payment gates, on-ch

Repository Structure

```
agentmesh/  
  apps/  
    web/    React canvas, builder UI, runtime console  
    api/    FastAPI control plane and agent orchestrator  
  packages/  
    specs/  Shared pipeline schema definitions  
  docs/  
    architecture.md  
    demo-flow.md
```

Key Backend Modules

Module	Path	Role
routes.py	app/api/routes.py	Public API surface — deploy, run, status endpoints
service.py	app/orchestrator/service.py	Main orchestration entry, routes to single or multi-agent runtime
executor.py	app/runtime/executor.py	Single-agent execution loop
orchestrator.py	app/a2a/orchestrator.py	Multi-agent A2A orchestrator, boots agent microservices
runtime.py	app/agents/runtime.py	Per-agent reasoning loop, tool selection, fan-out
protocol.py	app/a2a/protocol.py	Compiles canvas graph into runtime execution config
tools.py	app/runtime/tools.py	Tool execution — weather, search, crypto, chart, gmail
client.py	app/algorand/client.py	Algorand RPC, wallet creation, transactions, x402
repository.py	app/storage/repository.py	Local JSON persistence for pipelines and wallets

SECTION 3

The Canvas: Nodes and Wires

The canvas is not merely a visual representation. It is the execution graph. Every node and wire drawn by the user directly defines what the runtime is permitted to do during pipeline execution.

Node Types

Node	Role	Key Properties
Trigger	Pipeline entry point, activated by HTTP request	Node wallet, defines pipeline input schema
Agent	Autonomous reasoning unit with Algorand wallet	System prompt, LLM provider, connected tools and agents
API / Service	External tool or paid service endpoint	Tool kind (weather/search/crypto/chart/risk/gmail/custom), optional p
End	Terminal node, returns HTTP response to caller	Aggregates final output from upstream agents

Wire Types

Wire	Color	Protocol	Semantics
A2A	Purple	HTTP / A2A JSON-RPC	Agent-to-agent message passing. No on-chain payment. Enforces agent communication
x402	Green	HTTP + x402 payment	Agent pays a tool or service per call. Real ALGO deducted from agent wallet on-chain
ALGO Transfer	Blue	Algorand native transaction	Direct wallet-to-wallet ALGO movement. Used for escrow, settlement, and direct pay

The graph is the source of truth for execution permissions. An agent cannot message another agent unless a purple A2A wire connects them. An agent cannot call a tool unless a green x402 wire connects them. The backend enforces this topology strictly at runtime.

SECTION 4

Pipeline Lifecycle

Build

The user drags nodes onto the canvas, writes system prompts for agent nodes, configures tool nodes with pricing and endpoints, and connects everything with typed wires. Draft workflows are persisted in browser localStorage.

Deploy

On deploy, the backend generates Algorand keypairs for all agent nodes, persists the full pipeline definition and wallets in a local JSON repository, assigns a unique pipeline ID, and creates a live HTTP endpoint at POST /{pipeline_id}/run.

Fund

Each deployed agent node displays its Algorand wallet address in the UI. The user funds the wallet via copy, QR code, or direct faucet link. Agent wallets need ALGO to cover outgoing x402 tool payments and transaction fees.

Run

An external caller hits the pipeline endpoint. The x402 payment gate verifies the incoming payment. The backend selects single-agent or multi-agent execution based on graph topology. Agents execute, tools are called, A2A messages are exchanged, and logs stream back to the runtime console in real time.

Request Lifecycle

- **Step 1:** Frontend submits POST /{pipeline_id}/run
- **Step 2:** Backend loads the deployed pipeline definition from repository
- **Step 3:** x402 payment gate is checked — 402 response if unpaid
- **Step 4:** Backend selects single-agent or multi-agent runtime based on graph
- **Step 5:** Graph is compiled into runtime execution config (protocol.py)
- **Step 6:** Entry agent is triggered
- **Step 7:** Specialist agents and tools run in parallel where wires allow
- **Step 8:** Logs are collected from all agent runtimes
- **Step 9:** Final response is returned to the caller

SECTION 5

Single-Agent Execution

Single-agent execution is used when the pipeline graph contains only one agent node, or when no A2A wires are present. This path is optimized for simplicity and speed.

Execution Path

- Request enters routes.py
- PipelineOrchestrator.execute() loads the deployed workflow
- Graph is inspected — no A2A wires detected, single-agent path selected
- The analyzer agent is identified from the graph
- Agent inspects only its directly connected tool nodes
- Gemini planning or heuristic tool router selects the tool path
- Tool calls are executed with payment if priced
- Responder or End node returns the final output

Single-agent execution covers the majority of demo-ready use cases: market analysis, weather briefings, crypto lookups, and city research flows — all achievable with one agent and a set of connected tool nodes.

SECTION 6

Multi-Agent Execution

Multi-agent execution activates when the pipeline graph contains more than one agent node and at least one purple A2A wire. The orchestrator boots independent FastAPI microservices for each agent and routes messages between them according to the wire topology defined on the canvas.

Execution Steps

Graph Compilation. `protocol.py` converts the deployed workflow into a runtime config containing each agent's prompt, wallet, connected tools, and connected peer agents.

Agent Microservice Boot. `orchestrator.py` launches one FastAPI process per agent on a dedicated localhost port. Each agent exposes `/run`, `/message`, `/status`, and `/logs` endpoints.

Entry Agent Trigger. The orchestrator identifies the entry agent (connected to the Trigger node) and calls its `/run` endpoint with the pipeline input.

Parallel Fan-out. Inside `runtime.py`, an agent may call multiple connected specialist agents simultaneously using `asyncio.gather()`. This enables concurrent execution of independent sub-tasks.

Tool Execution. Each agent independently selects and executes only its connected tools, with payment fired for priced tools.

Downstream Synthesis. A downstream responder or summary agent may receive collected results from upstream agents and produce the final formatted output.

Log Aggregation. After execution, the orchestrator collects logs from all agent runtimes and returns them as a unified stream to the frontend console.

Parallel Execution Example

```
# Inside entry agent runtime (runtime.py)
results = await asyncio.gather(
    send_a2a_message(to='agent_technical', msg=market_data),
    send_a2a_message(to='agent_risk',      msg=market_data),
    x402_pay(url=oracle_url, amount=0.001)
)
# All three run concurrently – total time = max(individual times)
```


SECTION 7

Algorand Integration

Algorand is not a branding layer in AgentMesh. It is used throughout the runtime for wallet creation, balance management, payment execution, and on-chain audit anchoring.

Wallet Creation. On pipeline deploy, the backend generates a real Ed25519 keypair for each agent node using py-algorand-sdk. The address and private key are persisted in the local repository and remain stable across backend restarts.

Balance Polling. The frontend requests wallet balances via the backend, which fetches them live from the Algorand testnet via algod RPC. Balances display on agent nodes in the canvas UI and update after each run.

Tool Payments. For priced tool calls, the agent wallet signs and submits an Algorand payment transaction before the tool is executed. This uses native ALGO payments or the AVM x402 path depending on configuration.

x402 Payment Gate. The public pipeline endpoint checks for an incoming x402 payment header before execution begins. If absent, it returns a 402 response with facilitator metadata and expected payment details.

A2A Anchoring. When agents message each other, the orchestrator can anchor a 0-ALGO self-send transaction on Algorand with the A2A message hash as a transaction note. This creates an immutable on-chain audit trail.

Explorer Integration. Agent wallet addresses in the UI are linked to Lora testnet explorer URLs, allowing anyone to verify all transactions from a pipeline run.

Algorand Primitive	Usage in AgentMesh
Ed25519 Keypairs	Real agent wallet identity — one per agent node
Payment Transactions	Tool call payments and A2A anchoring
Atomic Transfers	Multi-leg pipeline settlements in one block
Box Storage	On-chain agent state and pipeline config persistence
Transaction Notes	A2A message hashes stored as on-chain audit log
x402 Protocol	HTTP-native micropayment gate on pipeline endpoint
Algod RPC	Live balance queries and transaction submission

SECTION 8

x402 Payment Protocol

x402 is an HTTP-native micropayment standard built on the long-dormant HTTP 402 Payment Required status code. It enables autonomous AI agents to pay for API access per request — no accounts, no subscriptions, no API keys. Algorand's x402 support reached full operational status in February 2026 via GoPlausible as the designated network facilitator.

How x402 Works in AgentMesh

- Agent attempts to call a priced tool or service endpoint
- Server returns HTTP 402 with payment metadata (amount, asset, receiver address)
- Agent's wallet signs and submits an Algorand payment transaction
- Payment is verified on-chain by the GoPlausible facilitator
- Server confirms payment and returns the tool response
- Agent wallet balance is decremented — visible on Lora explorer

x402 Flow Code

```
# Green wire execution – x402 payment + tool call
async def x402_pay(url: str, amount_algo: float, wallet_key: str):
    # Step 1: Preflight – get payment requirements
    r = await client.get(url)
    if r.status_code != 402:
        return r.json()
    meta = r.headers['X-Payment-Required']

    # Step 2: Sign and submit payment on Algorand
    txid = await algorand_client.pay(
        sender_key=wallet_key,
        receiver=meta['receiver'],
        amount_algo=amount_algo
    )

    # Step 3: Retry with payment proof
    r2 = await client.get(url, headers={'X-Payment-Response': txid})
    return r2.json()
```

Every green wire on the AgentMesh canvas maps directly to an x402 payment flow. The wire is not just visual — it is a payment relationship enforced at runtime. No wire, no payment, no access.

SECTION 9

Agent-to-Agent (A2A) Communication

Agent-to-agent communication in AgentMesh uses Google's A2A protocol — a JSON-RPC 2.0 message format transmitted over HTTP between agent microservices. Each agent is a running FastAPI server. Purple wires define which agents are permitted to communicate.

Message Format

```
# A2A message sent from one agent to another
{
  "jsonrpc": "2.0",
  "method": "tasks/send",
  "params": {
    "id": "task_uuid",
    "message": {
      "role": "user",
      "parts": [{ "type": "text", "text": "Analyze ALGO sentiment" }]
    },
    "metadata": {
      "from_agent": "agent_coordinator",
      "pipeline_id": "pm_x9k2"
    }
  }
}
```

Agent Microservice Endpoints

Endpoint	Method	Purpose
/run	POST	Orchestrator triggers entry agent to begin pipeline execution
/message	POST	Peer agents deliver A2A messages to this agent
/status	GET	Returns current agent state, wallet address, and balance
/logs	GET	Returns structured execution log for this agent's run

On-Chain Anchoring

Every A2A message exchange is optionally anchored on Algorand as a 0-ALGO self-send transaction with the message hash encoded in the transaction note field. This creates an immutable, verifiable audit trail of all inter-agent communication for a given pipeline run.

SECTION 10

Tool System

Tools are external capabilities that agents can invoke. They are represented as Service nodes on the canvas and connected to agents via green x402 wires. Tool selection within each agent is handled by a two-layer decision system.

Built-in Tool Types

Tool Kind	Data Source	Cost Model
weather	Open-Meteo API	Free or priced via x402
search	DuckDuckGo Instant Answer	Free or priced via x402
crypto	CoinGecko simple pricing	Free or priced via x402
chart	Market chart technical signal	Priced — x402 required
risk	Volatility-based risk model	Priced — x402 required
gmail	Gmail SMTP send/draft	Free (SMTP credentials required)
custom	Arbitrary URL pattern	Configurable — x402 optional

Tool Selection Logic

Tool selection operates through two layers. First, the Gemini planner receives the agent's task and connected tool list and selects the optimal tool path. If Gemini planning is unavailable or returns an ambiguous result, the fallback heuristic router in tools.py applies keyword and intent matching to select the most relevant tool.

SECTION 11

Security and Trust Model

Graph-Level Access Control. The pipeline execution graph is the primary access control mechanism. Agents can only call tools they are wired to. Agents can only message agents they are wired to. This is enforced by the backend at runtime — not advisory.

Wallet Key Management. Agent private keys are generated at deploy time and stored in the local JSON repository. For the hackathon implementation, keys are held in-process memory during execution. Production deployment would use OS keychain integration as implemented in the GoPlausible Algorand MCP.

Payment Verification. All x402 payments are verified on-chain via the GoPlausible facilitator before tool access is granted. Payments cannot be forged or replayed — each transaction ID is unique and verifiable on the Algorand testnet explorer.

A2A Message Integrity. A2A messages are anchored on Algorand with a SHA-256 hash of the message content. This allows any third party to verify that the logged message was not tampered with after the fact.

Pipeline Isolation. Each deployed pipeline has its own set of agent wallets, its own pipeline ID, and its own execution namespace. Pipelines cannot access each other's wallets or tool connections.

SECTION 12

Future Work

AgentMesh was built for the hackathon as a proof of concept demonstrating the full agentic commerce stack on Algorand. The following directions represent the natural product evolution:

AgentMesh SDK on PyPI. A Python SDK that lets any developer embed AgentMesh pipeline creation and execution in their own application — analogous to AlgoGate for x402 API paywalling.

Template Marketplace. A community library of pre-built pipeline templates — market analysis, trade signals, content workflows, hiring assistants — deployable in one click.

Mainnet Deployment. Migration from testnet to Algorand mainnet with real ALGO payments, production GoPlausible facilitator integration, and live pipeline execution.

Agent Reputation via Box Storage. Each agent accumulates a verifiable on-chain reputation score based on task completion history, stored in AVM Box Storage. High-reputation agents can charge premium prices.

Hosted Execution Runtime (SaaS). A managed cloud service where AgentMesh hosts the FastAPI agent runtimes on behalf of pipeline builders, removing the need for local server management.

Analytics Dashboard. Per-pipeline execution analytics — ALGO earned, ALGO spent, agent call counts, error rates, and average execution time — surfaced in the canvas UI.

AlgoKit Native Plugin. Integration with AlgoKit so developers can initialize AgentMesh pipelines directly from the AlgoKit CLI and scaffold agent nodes as AlgoKit smart contract templates.

Enterprise Multi-tenancy. Team workspaces, shared pipeline libraries, role-based access control, and audit log export for enterprise deployment.

REFERENCES

References & Links

Resource	URL
Algorand Python SDK	https://github.com/algorand/py-algorand-sdk
Algorand JS SDK	https://github.com/algorand/js-algorand-sdk
AlgoKit CLI	https://github.com/algorand/algokit-cli
GoPlausible x402 Facilitator	https://x402.goplausible.xyz
GoPlausible Algorand MCP	https://github.com/GoPlausible/algorand-mcp
x402 Protocol Specification	https://www.x402.org
x402 Whitepaper	https://www.x402.org/x402-whitepaper.pdf
Algorand x402 Page	https://algorand.co/agentic-commerce/x402
Google A2A Protocol	https://google.github.io/A2A
Lora Testnet Explorer	https://lora.algokit.io/testnet
Algorand Developer Docs	https://developer.algorand.org
GoPlausible MPP SDK	https://github.com/GoPlausible/algorand-mpp-sdk

AgentMesh — Build. Fund. Wire. Run.

Soubhagya Levi • UEM Kolkata • soubhagyalevi.me • github.com/notlevi911